

Reset Synchronizer (RST_SYNC)

Assignment 11 – Digital IC Design Course

Submitted by: Ziad Mostafa Abdelaziz

August 25, 2026

Contents

1	Introduction	2
2	Block Interface	2
2.1	Parameters	2
2.2	Ports Description	2
3	Architecture and Circuit Operation	2
3.1	Circuit Dynamics	3
4	RTL Verilog Module	3
5	Testbench and Simulation Results	4
5.1	Testbench Implementation	4
5.2	ModelSim Console Transcript	7
5.3	Output Waveform	7
5.4	Detailed Waveform and Signal Transition Analysis	7
5.4.1	Clock and Asynchronous Reset Input	8
5.4.2	2-Stage Synchronizer Operation (NUM_STAGES = 2)	8
5.4.3	4-Stage Synchronizer Operation (NUM_STAGES = 4)	8
6	SpyGlass Lint Results	8
6.1	Design Read Step Output	8
6.2	Lint RTL Goal Run Output	9
6.3	SpyGlass Lint Summary Report (moresimple.rpt)	10
7	Conclusion	10

1 Introduction

In synchronous digital IC design, un-synchronized active-low asynchronous resets present severe timing hazards during reset release (de-assertion). If an asynchronous reset transitions from active (0) to inactive (1) near a clock edge, it can violate the **recovery time** (t_{recovery}) or **removal time** (t_{removal}) requirements of destination flip-flops, causing metastability and uncoordinated state transitions.

The **Reset Synchronizer (RST_SYNC)** solves this timing hazard by providing:

1. **Immediate Asynchronous Reset Assertion:** Resets all destination flip-flops immediately when RST drops to 0 without waiting for a clock edge.
2. **Synchronous Reset De-assertion:** Releases the reset ($\text{SYNC_RST} \rightarrow 1$) strictly on a rising edge of CLK after NUM_STAGES clock cycles, preventing recovery/removal timing violations.

2 Block Interface

2.1 Parameters

Parameter Name	Description	Default Value
NUM_STAGES	Number of D Flip-Flop Synchronizer Stages	2

Table 1: Design Parameters

2.2 Ports Description

Signal Name	Description	Width
RST	Asynchronous active-low reset input	1
CLK	Destination domain clock	1
SYNC_RST	Synchronized active-low reset output	1

Table 2: Port Descriptions

3 Architecture and Circuit Operation

The RST_SYNC module consists of a parameterized chain of NUM_STAGES D flip-flops with active-low asynchronous reset inputs connected to the external RST line:

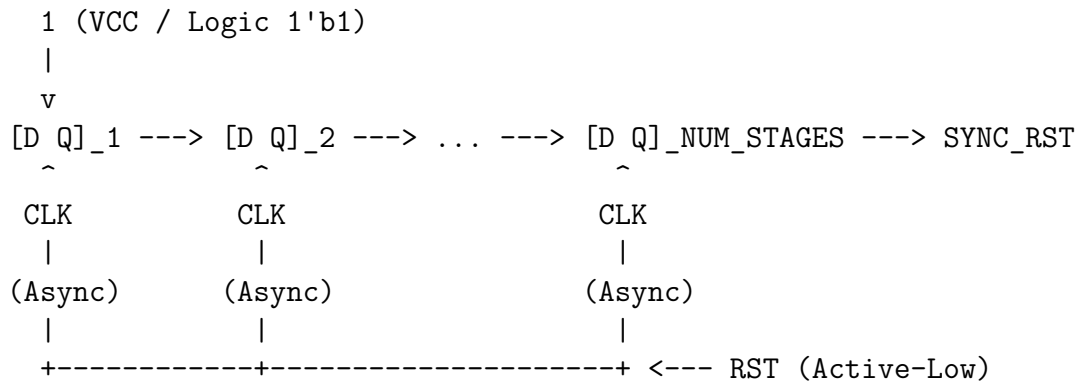


Figure 1: Reset Synchronizer Shift Register Architecture

3.1 Circuit Dynamics

- **Assertion Phase ($RST = 0$):** All flip-flops in the chain are asynchronously reset to 0. SYNC_RST drops to 0 immediately (< 1 ns delay).
- **De-assertion Phase ($RST = 1$):** The asynchronous reset is released. Constant high logic (1'b1) shifts through the chain on each **posedge** CLK. After NUM_STAGES clock edges, SYNC_RST transitions to 1 synchronously with CLK.

4 RTL Verilog Module

```

1 // =====
2 // Module Name   : RST_SYNC
3 // Description   : Synthesizable Reset Synchronizer for active-low asynchronous reset.
4 //               : Asynchronously asserts reset and synchronously de-asserts reset.
5 // Parameters    : NUM_STAGES - Number of D Flip-Flop stages (default = 2)
6 // Inputs        : RST       - Asynchronous active-low reset input
7 //               : CLK       - Destination domain clock
8 // Outputs       : SYNC_RST  - Synchronized active-low reset output
9 // =====
10
11 module RST_SYNC #(
12     parameter NUM_STAGES = 2
13 ) (
14     input wire RST,
15     input wire CLK,
16     output wire SYNC_RST
17 );
18
19 // Synchronizer Shift Register Chain
20 reg [NUM_STAGES-1:0] sync_reg;
21
22 // Asynchronous Reset Assertion & Synchronous De-assertion Logic
23 always @(posedge CLK or negedge RST) begin
24     if (!RST) begin
25         sync_reg <= {NUM_STAGES{1'b0}};
26     end else begin
27         sync_reg <= {sync_reg[NUM_STAGES-2:0], 1'b1};
28     end
29 end
30
31 // Synchronized Reset Output
32 assign SYNC_RST = sync_reg[NUM_STAGES-1];
33
34 endmodule

```

Listing 1: RST_SYNC.v – Synthesizable Reset Synchronizer RTL

5 Testbench and Simulation Results

5.1 Testbench Implementation

The self-checking testbench (RST_SYNC_tb.v) executes 4 comprehensive test cases simulating 100 MHz destination clock frequency ($T = 10$ ns).

```

1 `timescale 1ns/1ps
2
3 // =====
4 // Module Name   : RST_SYNC_tb
5 // Description   : Self-checking Testbench for RST_SYNC (Reset Synchronizer)
6 //               : Validates immediate asynchronous reset assertion and
7 //               : clock-synchronous reset de-assertion.
8 // =====
9
10 module RST_SYNC_tb;
11
12     // Clock Parameters
13     parameter CLK_PERIOD = 10.0; // 100 MHz (Period = 10 ns)
14
15     // Testbench Signals for Default DUT (NUM_STAGES = 2)
16     reg CLK;
17     reg RST;
18     wire SYNC_RST_2stage;
19
20     // Testbench Signals for 4-Stage DUT (NUM_STAGES = 4)
21     wire SYNC_RST_4stage;
22
23     // Verification Variables
24     integer err_count;
25
26     // Instantiate Default DUT (2 Stages)
27     RST_SYNC #(
28         .NUM_STAGES(2)
29     ) uut_2stage (
30         .RST      (RST),
31         .CLK       (CLK),
32         .SYNC_RST (SYNC_RST_2stage)
33     );
34
35     // Instantiate 4-Stage DUT (NUM_STAGES = 4)
36     RST_SYNC #(
37         .NUM_STAGES(4)
38     ) uut_4stage (
39         .RST      (RST),
40         .CLK       (CLK),
41         .SYNC_RST (SYNC_RST_4stage)
42     );
43
44     // -----
45     // Clock Generation
46     // -----
47     always # (CLK_PERIOD / 2.0) CLK = ~CLK;
48
49     // -----
50     // Main Test Execution Procedure
51     // -----
52     initial begin
53         // Initialize Signals
54         CLK      = 1'b0;
55         RST       = 1'b0; // Start in Reset

```

```

56     err_count = 0;
57
58     $display("=====");
59     $display("  RESET SYNCHRONIZER (RST_SYNC) SIMULATION TESTBENCH  ");
60     $display("  Clock Frequency: 100 MHz (Period = 10 ns)          ");
61     $display("=====");
62
63     // -----
64     // TEST CASE 1: Initial Reset Status Check
65     // -----
66     #5;
67     $display("\n--- TC1: Initial Reset Status Check ---");
68     if (SYNC_RST_2stage != 1'b0 || SYNC_RST_4stage != 1'b0) begin
69         $display("  -> ERROR: SYNC_RST not zero during active reset!");
70         err_count = err_count + 1;
71     end else begin
72         $display("  -> SUCCESS: SYNC_RST is zero during active reset.");
73     end
74
75     // -----
76     // TEST CASE 2: Synchronous De-assertion Test (2 Stages)
77     // -----
78     $display("\n--- TC2: Synchronous De-assertion Test (NUM_STAGES = 2) ---");
79     test_deassertion_2stage();
80
81     // -----
82     // TEST CASE 3: Immediate Asynchronous Assertion Test
83     // -----
84     $display("\n--- TC3: Immediate Asynchronous Reset Assertion Test ---");
85     test_async_assertion();
86
87     // -----
88     // TEST CASE 4: Parameter Overriding Test (NUM_STAGES = 4)
89     // -----
90     $display("\n--- TC4: Synchronous De-assertion Test (NUM_STAGES = 4) ---");
91     test_deassertion_4stage();
92
93     // -----
94     // FINAL SUMMARY
95     // -----
96     $display("\n=====");
97     if (err_count == 0)
98         $display("  ALL TEST CASES PASSED SUCCESSFULLY! (0 Errors)");
99     else
100         $display("  SIMULATION FAILED WITH %0d ERRORS!", err_count);
101     $display("=====");
102     #50;
103     $stop;
104 end
105
106 // -----
107 // Verification Tasks
108 // -----
109
110 // TC2 Task: Test De-assertion for 2-stage synchronizer
111 task test_deassertion_2stage;
112 begin
113     // Release RST asynchronously mid-clock-cycle (at offset +3.5 ns after negedge)
114     @(negedge CLK);
115     #3.5;
116     RST = 1'b1; // Asynchronous de-assertion
117     $display("  -> RST de-asserted asynchronously to 1 mid-cycle");
118
119     // Verify SYNC_RST remains 0 before clock edge
120     if (SYNC_RST_2stage != 1'b0) begin
121         $display("  -> ERROR: SYNC_RST de-asserted before clock edge!");
122         err_count = err_count + 1;
123     end
124
125     // 1st posedge CLK: sync_reg becomes 2'b01 (SYNC_RST = 0)
126     @(posedge CLK);

```

```

127     #1;
128     if (SYNC_RST_2stage !== 1'b0) begin
129         $display(" -> ERROR: SYNC_RST de-asserted after only 1 cycle (exp 2)!");
130         err_count = err_count + 1;
131     end else begin
132         $display(" -> Verified 1st clock edge: SYNC_RST stays 0 (1st stage captured)");
133     end
134
135     // 2nd posedge CLK: sync_reg becomes 2'b11 (SYNC_RST = 1)
136     @(posedge CLK);
137     #1;
138     if (SYNC_RST_2stage !== 1'b1) begin
139         $display(" -> ERROR: SYNC_RST failed to de-assert on 2nd clock edge!");
140         err_count = err_count + 1;
141     end else begin
142         $display(" -> SUCCESS: SYNC_RST de-asserted to 1 synchronously on 2nd clock edge!");
143     end
144
145     repeat(2) @(posedge CLK);
146 end
147 endtask
148
149 // TC3 Task: Test Immediate Asynchronous Assertion
150 task test_async_assertion;
151 begin
152     // Assert RST asynchronously mid-cycle
153     @(posedge CLK);
154     #2.5; // Mid-cycle offset
155     RST = 1'b0; // Asynchronous assertion
156     #0.5; // Less than 1 ns check
157
158     if (SYNC_RST_2stage !== 1'b0) begin
159         $display(" -> ERROR: Asynchronous reset assertion failed to drop SYNC_RST
160             immediately!");
161         err_count = err_count + 1;
162     end else begin
163         $display(" -> SUCCESS: SYNC_RST dropped to 0 immediately (<1 ns) upon async RST
164             assertion.");
165     end
166
167     repeat(2) @(posedge CLK);
168 end
169 endtask
170
171 // TC4 Task: Test De-assertion for 4-stage synchronizer
172 task test_deassertion_4stage;
173 integer cycle;
174 begin
175     @(negedge CLK);
176     #3.5;
177     RST = 1'b1;
178
179     for (cycle = 1; cycle <= 3; cycle = cycle + 1) begin
180         @(posedge CLK);
181         #1;
182         if (SYNC_RST_4stage !== 1'b0) begin
183             $display(" -> ERROR: 4-stage SYNC_RST de-asserted prematurely at cycle %0d!",
184                 cycle);
185             err_count = err_count + 1;
186         end
187     end
188
189     // 4th posedge CLK: SYNC_RST_4stage should de-assert
190     @(posedge CLK);
191     #1;
192     if (SYNC_RST_4stage !== 1'b1) begin
193         $display(" -> ERROR: 4-stage SYNC_RST failed to de-assert on 4th clock edge!");
194         err_count = err_count + 1;
195     end else begin
196         $display(" -> SUCCESS: 4-stage SYNC_RST de-asserted to 1 synchronously on 4th

```

```

194         end
195
196         repeat (2) @(posedge CLK);
197     end
198 endtask
199
200 endmodule

```

Listing 2: RST_SYNC_tb.v – Self-Checking Testbench

5.2 ModelSim Console Transcript

All 4 test cases passed successfully with 0 errors:

```

=====
| RESET SYNCHRONIZER (RST_SYNC) SIMULATION TESTBENCH
| Clock Frequency: 100 MHz (Period = 10 ns)
| =====
|
| --- TC1: Initial Reset Status Check ---
| -> SUCCESS: SYNC_RST is zero during active reset.
|
| --- TC2: Synchronous De-assertion Test (NUM_STAGES = 2) ---
| -> RST de-asserted asynchronously to 1 mid-cycle
| -> Verified 1st clock edge: SYNC_RST stays 0 (1st stage captured)
| -> SUCCESS: SYNC_RST de-asserted to 1 synchronously on 2nd clock edge!
|
| --- TC3: Immediate Asynchronous Reset Assertion Test ---
| -> SUCCESS: SYNC_RST dropped to 0 immediately (<1 ns) upon async RST assertion.
|
| --- TC4: Synchronous De-assertion Test (NUM_STAGES = 4) ---
| -> SUCCESS: 4-stage SYNC_RST de-asserted to 1 synchronously on 4th clock edge!
|
| =====
| ALL TEST CASES PASSED SUCCESSFULLY! (0 Errors)
| =====

```

Figure 2: ModelSim Transcript Log – All 4 Test Cases PASSED (0 Errors)

5.3 Output Waveform

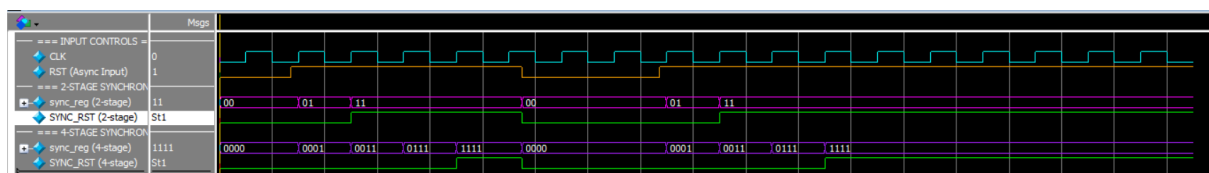


Figure 3: ModelSim Output Waveform – RST_SYNC Simulation

5.4 Detailed Waveform and Signal Transition Analysis

The output waveform captures the dynamic behavior of both 2-stage and 4-stage reset synchronizers:

5.4.1 Clock and Asynchronous Reset Input

- **CLK (Cyan):** Destination domain clock operating at $f = 100$ MHz ($T = 10$ ns).
- **RST (Orange, Async Input):** Asynchronous active-low reset input.
 - At initial time $t = 0$, $RST = 0$ (active reset).
 - At $t \approx 25$ ns, RST is de-asserted to 1 asynchronously mid-cycle.
 - At $t \approx 75$ ns, RST is re-asserted to 0 asynchronously mid-cycle.
 - At $t \approx 105$ ns, RST is de-asserted to 1 again mid-cycle.

5.4.2 2-Stage Synchronizer Operation ($NUM_STAGES = 2$)

- **sync_reg (Magenta, Binary):**
 1. During active reset ($RST = 0$), $sync_reg = 2'b00$.
 2. When RST de-asserts to 1 mid-cycle, $sync_reg$ stays $2'b00$ until the first clock edge.
 3. On 1st posedge CLK: Stage 1 captures 1, so $sync_reg \rightarrow 2'b01$. Output $SYNC_RST$ stays 0.
 4. On 2nd posedge CLK: Stage 2 captures 1, so $sync_reg \rightarrow 2'b11$. Output $SYNC_RST$ de-asserts to 1 **strictly on the clock edge**.
- **SYNC_RST_2stage (Lime):** Remains 0 during the 2-cycle synchronization latency, then rises to 1 synchronously on the 2nd rising clock edge. When RST drops to 0 asynchronously, $SYNC_RST_2stage$ drops to 0 **immediately** (< 1 ns).

5.4.3 4-Stage Synchronizer Operation ($NUM_STAGES = 4$)

- **sync_reg (Purple, Binary):** Shifts $1'b1$ sequentially through 4 flip-flops:

$$4'b0000 \rightarrow 4'b0001 \rightarrow 4'b0011 \rightarrow 4'b0111 \rightarrow 4'b1111$$
- **SYNC_RST_4stage (Green):** De-asserts to 1 on the 4th rising clock edge (40 ns latency), confirming that NUM_STAGES parameterization correctly scales the de-assertion latency.

6 SpyGlass Lint Results

The design was analyzed using Synopsys SpyGlass (version vL-2016.06) under the `rtl_handoff` methodology.

6.1 Design Read Step Output

SpyGlass successfully read and elaborated `RST_SYNC.v`. The message tree shows **Total: 2 messages, Displayed: 2, Waived: 0** – both are informational messages.

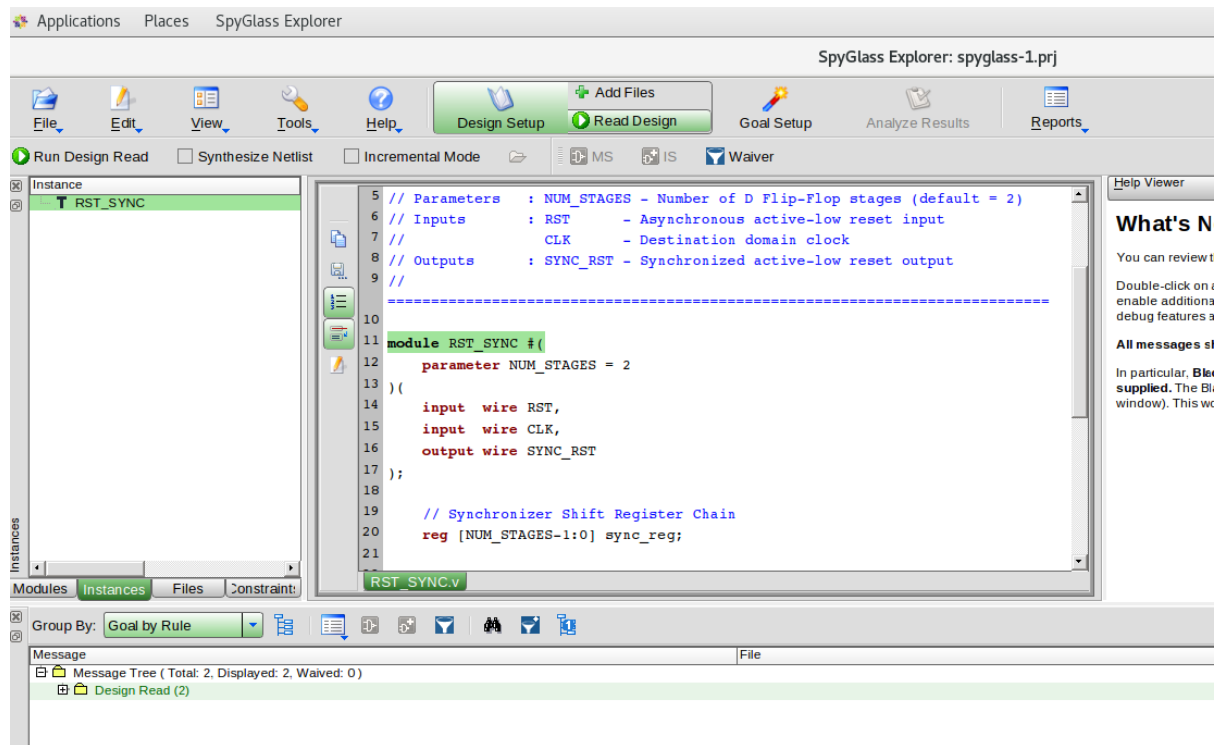


Figure 4: SpyGlass Design Read Step Output – 2 Info Messages, 0 Errors

6.2 Lint RTL Goal Run Output

The Lint RTL goal run completed with a **clean lint result** (0 Errors, 0 Warnings, 0 Latches).

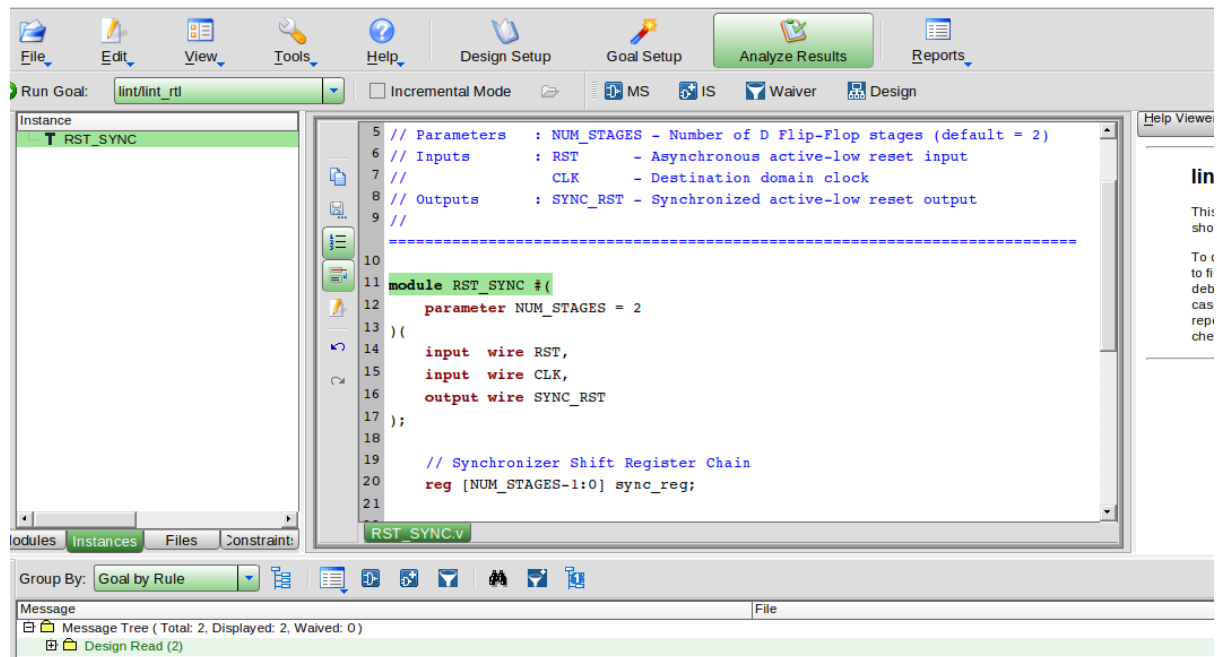


Figure 5: SpyGlass Lint RTL Goal Run Output – Clean Lint (0 Errors, 0 Warnings)

6.3 SpyGlass Lint Summary Report (moresimple.rpt)

1	Total Number of Generated Messages :	2
2	Number of Waived Messages :	0
3	Number of Reported Messages :	2
4	Number of Overlimit Messages :	0
5		
6	MORESIMPLE REPORT:	
7		
8	##### BuiltIn -> RuleGroup=Design Read #####	
9	[1] DetectTopDesignUnits Info	../rtl/RST_SYNC.v Line 11
10	Module RST_SYNC is a top level design unit	
11	[0] ElabSummary Info	
12	Please refer file elab_summary.rpt for elab summary report	

Listing 3: SpyGlass moresimple.rpt – Lint Summary Report

7 Conclusion

The Reset Synchronizer (RST_SYNC) module was successfully designed, modeled, and verified.

- Immediate asynchronous reset assertion guarantees rapid system protection (< 1 ns).
- Synchronous reset de-assertion eliminates recovery and removal timing violations across the destination clock domain.
- Parameterized flip-flop stage depth (NUM_STAGES) allows configurable synchronization latency.
- All 4 test cases passed with 0 errors in ModelSim simulation.
- Clean SpyGlass lint report: 0 errors, 0 warnings, 0 latches.